Fatec Sorocaba "José Crespo Gonzales"

Centro Paula Souza

SÃO PAULO GOVERNO DO ESTADO

Secretaria de Desenvolvimento Econômico

Programação para WEB

4º Semestre **Turno: manhã**

Prof.ª Denilce de Almeida Oliveira Veloso **Data: 23/02/2026**

Nome: Eduardo Ferreira Proença

FERRAMENTAS DE TESTE INCLUIR NOÇÕES DE TESTES AUTOMATIZADOS NO FRONT E BACK-END

A complexidade crescente das aplicações, no back-end exige abordagens robustas para identificar e corrigir falhas precocemente. Nesse contexto, os testes automatizados emergem como uma prática indispensável, permitindo que as equipes de desenvolvimento validem o comportamento de seus sistemas de forma eficiente e contínua.

A adoção de testes automatizados não apenas acelera o ciclo de desenvolvimento, mas também aumenta a confiança no código, reduzindo o risco de regressões e garantindo que as funcionalidades entregues atendam às expectativas dos usuários .

O uso de ferramentas para teste automatizado permite aumentar a eficiência e a cobertura dos testes, reduzindo falhas e garantindo um software mais robusto. Com a diversidade de aplicações e tecnologias disponíveis no mercado, escolher a ferramenta certa pode fazer toda a diferença no processo de desenvolvimento. A seguir, conheça algumas das principais ferramentas utilizadasriar uma ponte e compartilhar algo com outra pessoa.

JEST

Jest é um framework de testes versátil que também encontra vasta aplicação no desenvolvimento back-end, especialmente no ecossistema Node.js. Quando combinado com bibliotecas como o Supertest, o Jest se torna uma ferramenta poderosa para testar rotas de API e a lógica de negócio implementada no back-end . O Supertest facilita a simulação de requisições HTTP para endpoints da API, permitindo que os desenvolvedores escrevam testes que validam o comportamento das rotas, a autenticação, a autorização e a manipulação de dados .

A combinação de Jest e Supertest é particularmente eficaz para testes unitários e de integração de APIs, onde a velocidade e a capacidade de isolar componentes são cruciais. Essa abordagem permite que os desenvolvedores testem a lógica do servidor sem a necessidade de implantar a aplicação em um ambiente completo, proporcionando um feedback rápido e eficiente durante o ciclo de desenvolvimento.

CYPRESS

O Cypress é especificamente para testes E2E e de integração. Sua principal característica é a execução dos testes diretamente no navegador, o que proporciona uma experiência de depuração em tempo real e uma visualização clara do que está acontecendo durante a execução dos testes . A interface gráfica (GUI) do Cypress é um de seus grandes diferenciais, permitindo aos desenvolvedores observar cada passo do teste, inspecionar elementos, comandos e o estado da aplicação em qualquer ponto da execução. Isso facilita a identificação e correção de problemas de forma ágil .

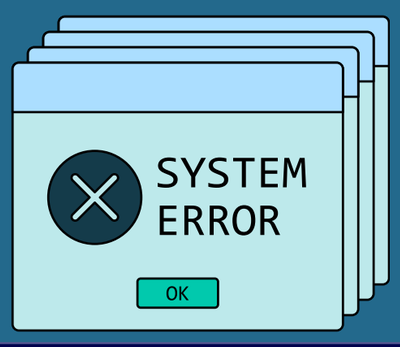
Embora o Cypress seja altamente eficaz para simular interações do usuário e validar fluxos complexos, ele apresenta uma desvantagem notável em termos de velocidade quando comparado a outras ferramentas como o Playwright. Sua arquitetura, que envolve a execução no navegador, pode torná-lo mais lento para grandes suítes de testes, especialmente em cenários de integração contínua .

PLAYWRIGHT

Embora o Playwright seja amplamente conhecido por sua capacidade de automação de navegadores, ele também oferece uma API de requisições integrada que o torna uma ferramenta eficaz para testes de API . Essa funcionalidade permite que os desenvolvedores realizem requisições HTTP diretamente, sem a necessidade de iniciar uma instância de navegador, o que o torna uma alternativa rápida e eficiente para testar endpoints de back-end.

O Playwright para testes de API se beneficia de sua arquitetura leve e da capacidade de configurar opções como retries, paralelismo e timeouts flexíveis, características que são valiosas em cenários de teste de API .
Essa capacidade de realizar testes de API com o Playwright é particularmente útil em projetos onde já se utiliza a ferramenta para testes de front-end, permitindo consolidar a estratégia de testes em uma única plataforma.
A performance do Playwright em testes de API é comparável à do Jest, sendo significativamente mais rápido que o Cypress para essa finalidade

CONCLUSÃO

A escolha das ferramentas de teste automatizado é um passo crítico para garantir a qualidade e a sustentabilidade de projetos de software. Como demonstrado, a adoção de uma estratégia de testes que abranja desde os testes unitários até os testes de ponta a ponta, seguindo a pirâmide de testes, é fundamental para um ciclo de desenvolvimento eficiente e confiável. Ferramentas como Jest, Cypress e Playwright oferecem soluções robustas e complementares para atender às diversas necessidades de teste em ambientes front-end e back-end.

ALURA. Tipos de testes: quais os principais e por que utilizá-los? Disponível em: https://www.alura.com.br/artigos/tipos-de-testes-principais-por-que-utiliza-los. Acesso em: 23 fev. 2026.
ENIN, Andrey. API Testing Comparison: Cypress vs. Playwright vs. Jest. JavaScript in Plain English, 2023. Disponível em: https://javascript.plainenglish.io/api-testing-comparison-cypress-vs-playwright-vs-jest-2ff1f80c5a7b. Acesso em: 23 fev. 2026.
POSTMAN. Frontend and API Testing Using Cypress and Postman. Postman Blog, 2022. Disponível em: https://blog.postman.com/front-end-api-testing-cypress-postman/. Acesso em: 23 fev. 2026.